

Bootstrap e CrUD: Insert, Delete e Update



Prof. Dr. João Paulo Lemos Escola
Copyright© 2026

O que veremos?

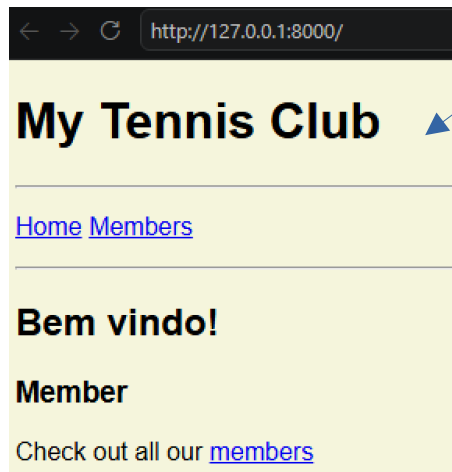
- Integração com Bootstrap;
- Formulários de CRUD;
- Botões de deletar e alterar (FontAwesome).

Sumário

- 1) Integrar com Bootstrap;
- 2) Cadastro de membros (insert);
- 3) Delete;
- 4) Update.

Antes de começar...

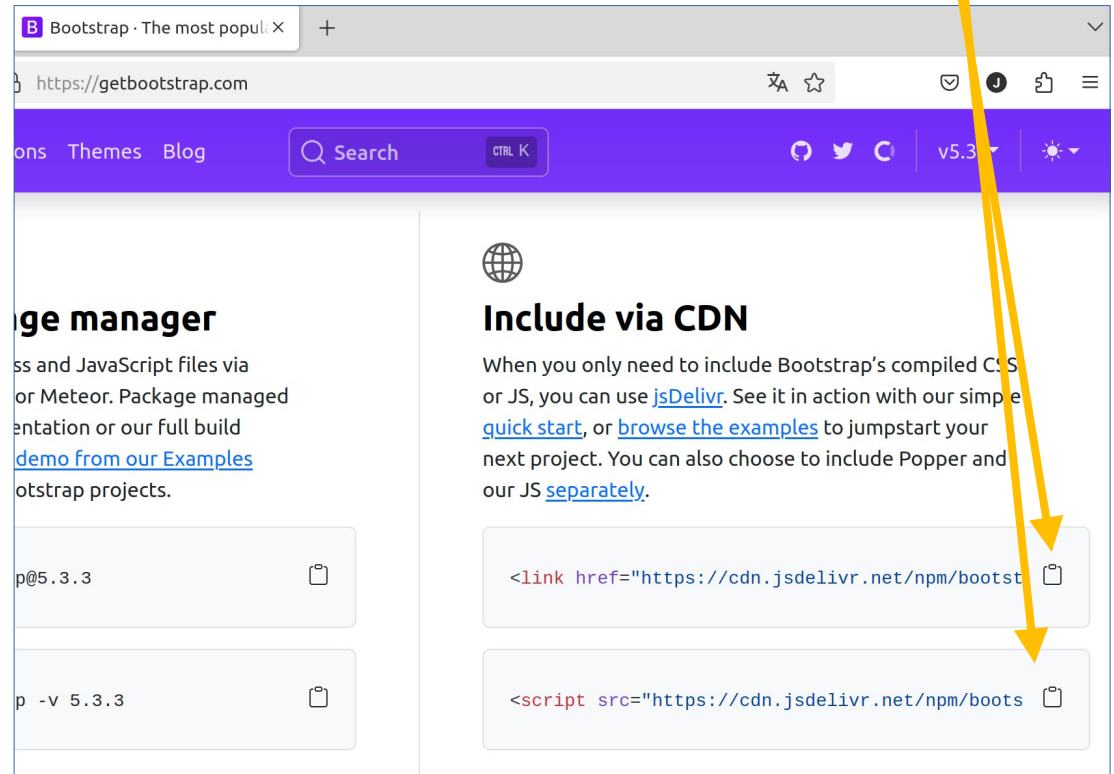
- Baixe o projeto https://github.com/jpescola/my_tennis_club_aula3
- Descompacte o zip;
- Renomeie a pasta para **my_tennis_club_aula4**;
- Entre no cmd e digite os comandos:
 - `py -m venv .venv`
 - `.venv\scripts\activate`
 - `pip install django`
 - `py manage.py runserver`



1) Integrar com Bootstrap

Clique para copia o link

Vamos iniciar buscando o CDN do Bootstrap; Em seguida o incluiremos no arquivo 'master.html' para ser replicado por todas as páginas do módulo:



1º passo

- Instalar o Bootstrap no projeto:
 - Para isso, vamos incluir o CDN no elemento 'HEAD' do arquivo principal do nosso projeto.



```
members > templates > master.html > ...
1 <!doctype html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <title>My Tennis Club {% block title %}{% endblock %}</title>
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" rel="stylesheet"
9     integrity="sha384-GLhlTQ8iRABdZLL603oVMWSktQ0p6b7In1Zl3/Jr59b6EGGoI1aFkw7cmDA6j6gD" crossorigin="anonymous">
10 </head>
11
```

script no final do body

Package manager

Install Bootstrap's CSS and JavaScript files via npm, Yarn, or Meteor. Package managed installation or our full build system are also available. [demo from our Examples](#)

```
npm install bootstrap@5.3.3
```

```
npm install bootstrap@5.3.3
```

Include via CDN

When you only need to include Bootstrap's compiled CSS or JS, you can use [jsDelivr](#). See it in action with our simple [quick start](#), or [browse the examples](#) to jumpstart your next project. You can also choose to include Popper and our JS [separately](#).

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" integrity="sha384-w76AqPfDkMBDXo30jS1SgeZ6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwXhTBTkF7CXvN" crossorigin="anonymous">
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/js/bootstrap.bundle.min.js" integrity="sha384-w76AqPfDkMBDXo30jS1SgeZ6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwXhTBTkF7CXvN" crossorigin="anonymous"></script>
```

```
19
20 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/js/bootstrap.bundle.min.js"
21     integrity="sha384-w76AqPfDkMBDXo30jS1SgeZ6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwXhTBTkF7CXvN"
22     crossorigin="anonymous"></script>
23 </body>
24
25 </html>
```

2º passo

- Vamos adicionar um navbar, uma barra de navegação (menu);
- Para isso, crie o arquivo 'navbar.html' com o código disponível na documentação:

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>



https://getbootstrap.com/docs/5.3/getting-started/introduction/



Docs Examples Icons Themes Blog

Search

CTRL K



Getting started

Introduction

Download

Contents

Browsers & devices

JavaScript

Webpack

Parcel

Vite

Accessibility

RFS

RTL

Contribute

Get started with Bootstrap

[View on GitHub](#)

Bootstrap is a powerful, feature-packed frontend toolkit.
Build anything—from prototype to production—in minutes.



Design and Development tips in your inbox. Every weekday.

ads via Carbon

Encontre o link 'Navbar'

getbootstrap.com/docs/5.3/components/navbar/

Docs Examples Icons Blog Search

Components

- Accordion
- Alerts
- Badge
- Breadcrumb
- Buttons
- Button group
- Card
- Carousel
- Close button
- Collapse
- Dropdowns
- List group
- Modal
- Navbar**
- Navs & tabs
- Offcanvas
- Pagination
- Placeholders
- Popovers
- Progress
- Scrollspy
- Spinners
- Toasts
- Tooltips

Helpers

- Clearfix
- Color & background

Navbar Home Features Pricing Dropdown link ▾

HTML

```
<nav class="navbar navbar-expand-lg bg-body-tertiary">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">Navbar</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="navbarNavDropdown">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a class="nav-link active" aria-current="page" href="#">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="#">Features</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" href="#">Pricing</a>
      </li>
      <li class="nav-item dropdown">
        <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdo
        Dropdown link
        </a>
        <ul class="dropdown-menu">
          <li><a class="dropdown-item" href="#">Action</a></li>
          <li><a class="dropdown-item" href="#">Another action</a></li>
          <li><a class="dropdown-item" href="#">Something else here</a></li>
        </ul>
      </li>
    </ul>
  </div>
</div>
</div>
</nav>
```

Clique para copiar o código e cole no arquivo navbar.html

```
1 |
2 <nav class="navbar navbar-expand-lg bg-body-tertiary">
3   <div class="container-fluid">
4
5     <a class="navbar-brand" href="#">
6
7       <h1>My Tennis Club</h1>
8
9     </a>
10
11     <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav"
12       aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
13       <span class="navbar-toggler-icon"></span>
14     </button>
15
16     <div class="collapse navbar-collapse" id="navbarNav">
17       <ul class="navbar-nav">
18
19         <li class="nav-item">
20           <a class="nav-link active" aria-current="page" href="/">Home</a>
21         </li>
22
23         <li class="nav-item">
24           <a class="nav-link" href="/members/">members</a>
25         </li>
26
27       </ul>
28     </div>
29
30   </div>
31 </nav>
```

Substitua os
links

3º passo

- Adicione um elemento 'container' no master.html;
- Inclua o 'navbar' no master.html.

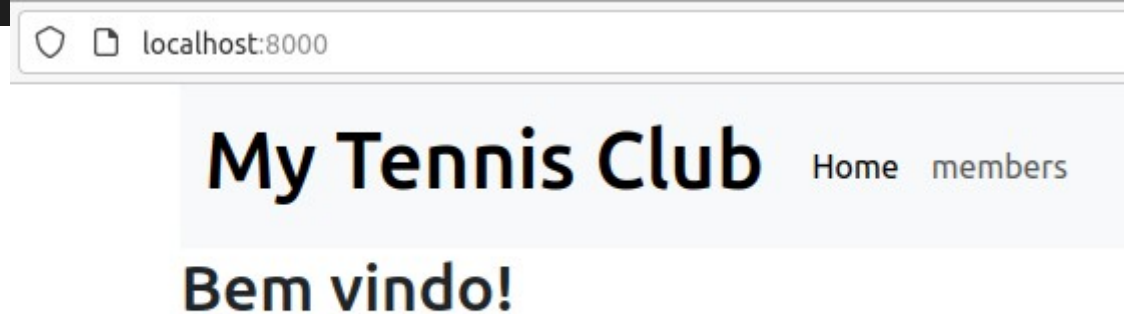
members > templates > master.html > ...

```
1  <!doctype html>
2  <html lang="en">
3
4  <head>
5      <meta charset="utf-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1">
7      <title>My Tennis Club {% block title %}{% endblock %}</title>
8      <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" rel="stylesheet"
9          integrity="sha384-GLhlTQ8iRABdZLL603oVMW/SktQOp6b7In1Zl3/Jr59b6EGGoI1aFkw7cmDA6j6gD" crossorigin="anonymous">
10 </head>
11
12 <body>
13     <div class="container">
14         {% include 'navbar.html' %}
15
16         {% block content %}
17         {% endblock %}
18     </div>
19
20     <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/js/bootstrap.bundle.min.js"
21         integrity="sha384-w76AqPfDkMBDXo30jS1SgeZ6pr3x5MlQ1ZAGC+nuZB+EYdgRZgiwxBTbTf7CXvN"
22         crossorigin="anonymous"></script>
23 </body>
24
25 </html>
```

Testando

- Confira o main.html:

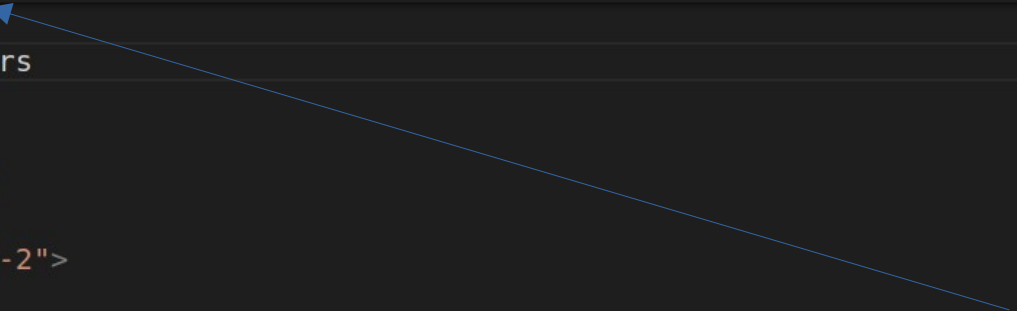
```
members > templates > < main.html > ...  
1 {% extends "master.html" %}  
2  
3 {% block content %}  
4 <h2>Bem vindo!</h2>  
5 {% endblock %}
```



```
1  {% extends "master.html" %}
2
3  {% block title %}
4  - Detalhes de {{ mymember.firstname }} {{ mymember.lastname }}
5  {% endblock %}
6
7  {% block content %}
8
9  <div class="card mt-2">
10
11      <div class="card-header">
12          Details about {{ mymember.firstname }} {{ mymember.lastname }}
13      </div>
14      <div class="card-body">
15
16          <h1>{{ mymember.firstname }} {{ mymember.lastname }}</h1>
17
18          <p>Phone {{ mymember.phone }}</p>
19          <p>Member since: {{ mymember.joined_date }}</p>
20
21          <p><a href="/members">Voltar</a></p>
22
23      </div>
24  </div>
25
26  {% endblock %}
```

members > templates > all_members.html > ...

```
3 {% block title %}  
4 - List of all members  
5 {% endblock %}  
6  
7 {% block content %}  
8  
9 <div class="card mt-2">  
10  
11     <div class="card-header">  
12         Members  
13     </div>  
14  
15     <div class="card-body">  
16  
17         <ul class="list-group">  
18             {% for x in mymembers %}  
19                 <li class="list-group-item"><a href="details/{{ x.id }}">{{ x.firstname }} {{ x.lastname }}</a></li>  
20             {% endfor %}  
21         </ul>  
22  
23     </div>  
24  
25 </div>  
26  
27 {% endblock %}
```



Testando...

127.0.0.1:8000/members/

My Tennis Club

Home members

Members

- [Emil Refsnes](#)
- [Tobias Refsnes](#)
- [Linus Refsnes](#)
- [Lene Refsnes](#)
- [Stalikken Refsnes](#)

127.0.0.1:8000/members/details/1

My Tennis Club

Home members

Details about Emil Refsnes

Emil Refsnes

Phone 5551234

Member since: Jan. 5, 2022

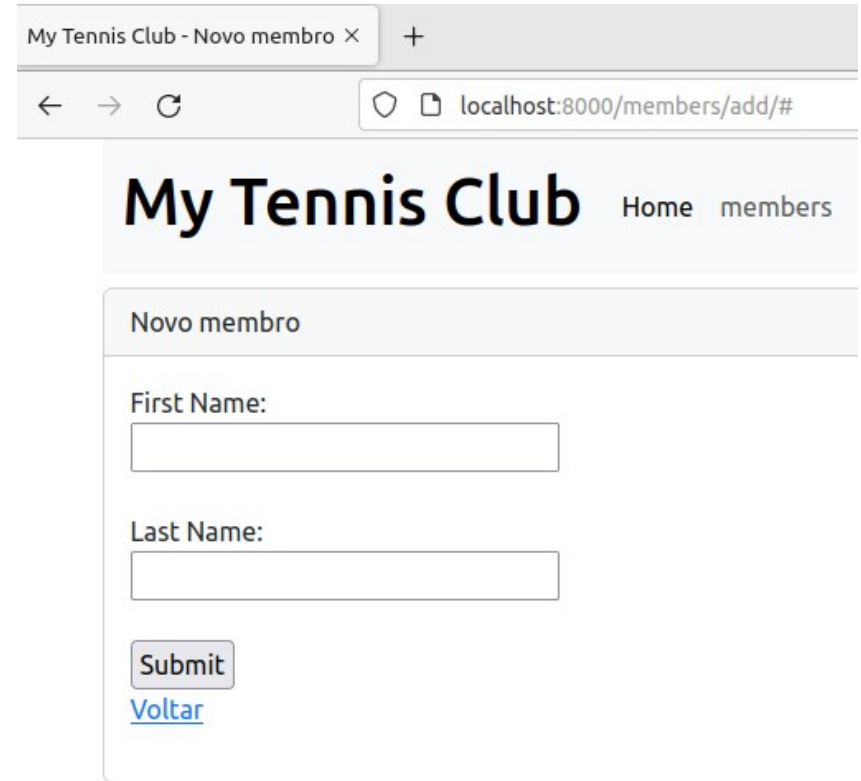
[Voltar](#)

Continuação do CRUD

- Nas aulas anteriores vimos como manipular o BD usando o console (terminal de comandos);
- Também já vimos como listar dados em páginas Web (R do cRud);
- Agora vamos aprender as demais tarefas do CRUD: criar (C), atualizar (U) e deletar (D) registros.

2) Cadastro de membros

- Vamos criar um formulário de **cadastro** de membros:



The screenshot shows a web browser window with the title 'My Tennis Club - Novo membro'. The address bar displays 'localhost:8000/members/add/#'. The page header includes 'My Tennis Club' and navigation links 'Home' and 'members'. The main content area is titled 'Novo membro' and contains a registration form with two text input fields labeled 'First Name:' and 'Last Name:'. Below the fields are a 'Submit' button and a blue link labeled 'Voltar'.

My Tennis Club - Novo membro × +

← → ↺ localhost:8000/members/add/#

My Tennis Club Home members

Novo membro

First Name:

Last Name:

[Voltar](#)

add.html

```
members > templates > <> add.html
1  {% extends "master.html" %}
2
3  {% block title %}
4    - Novo membro
5  {% endblock %}
6
7  {% block content %}
8
9    <div class="card mt-2">
10
11      <div class="card-header">
12        Novo membro
13      </div>
14      <div class="card-body">
15
16        <form action="addrecord/" method="post">
17          {% csrf_token %}
18          |
19          First Name:<br>
20          <input name="first">
21          <br><br>
22          Last Name:<br>
23          <input name="last">
24          <br><br>
25          <input type="submit" value="Submit">
26        </form>
27
28        <p><a href="/members">Voltar</a></p>
29      </div>
30    </div>
31  </div>
32
33  {% endblock %}
```

Parâmetros
que serão
recebidos
posteriormente
pela view
addrecord

csrf_token

- Cross Site Request Forgeries
- Recurso de segurança do Django
- Utilizados em forms do tipo POST

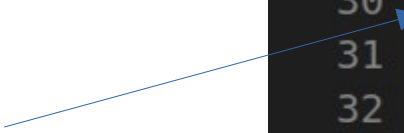
```
15
16     <form action="addrecord/" method="post">
17         {% csrf_token %}
18         |
19         First Name:<br>
```



members/views.py

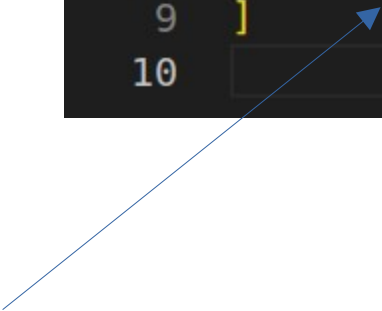
- Essa view vai apresentar o formulário novo;
- Para isso, já criamos o arquivo *add.html*

```
members > views.py > addrecord  
  
29  
30 def add(request):  
31     template = loader.get_template('add.html')  
32     return HttpResponse(template.render({}, request))  
33
```



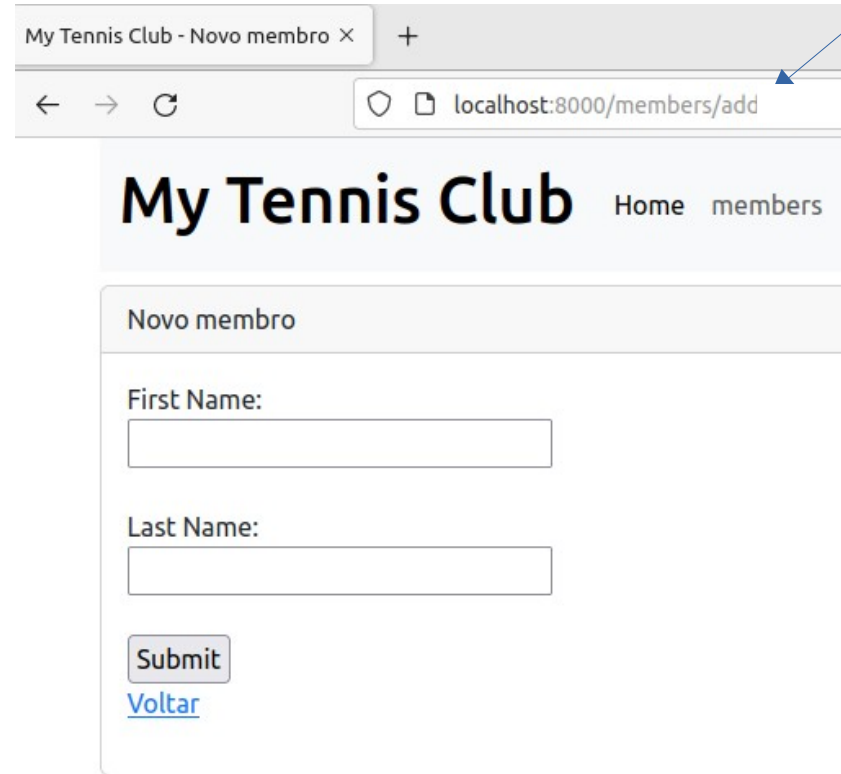
Inclua a nova URL

```
members > urls.py > ...  
2  from . import views  
3  
4  urlpatterns = [  
5      path('', views.main, name='main'),  
6      path('members/', views.members, name='members'),  
7      path('members/details/<int:id>', views.details, name='details'),  
8      path('members/add/', views.add, name='add'),  
9  ]  
10
```



Veja o resultado

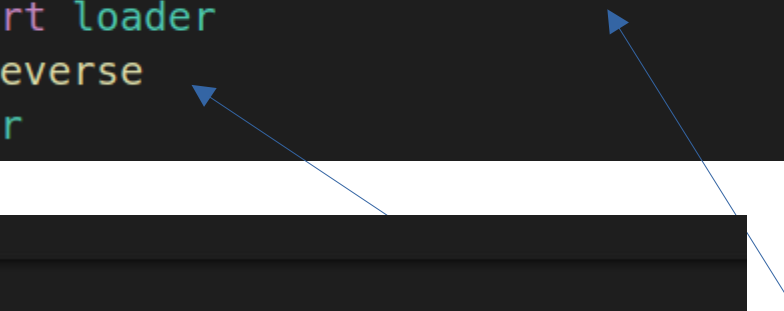
- Ainda **não** vai funcionar, pois falta configurar o procedimento de gravação (salvamento) do objeto:



The screenshot shows a web browser window with the title 'My Tennis Club - Novo membro'. The address bar displays 'localhost:8000/members/add'. The page content includes a header for 'My Tennis Club' with navigation links 'Home' and 'members'. Below this is a section titled 'Novo membro' containing a form with two text input fields labeled 'First Name:' and 'Last Name:'. At the bottom of the form are a 'Submit' button and a blue link labeled 'Voltar'. A blue arrow points from the top right of the image towards the browser window.

members > views.py > ...

```
1 from django.http import HttpResponseRedirect
2 from django.template import loader
3 from django.urls import reverse
4 from .models import Member
```



members > views.py > addrecord

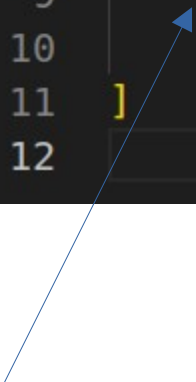
```
29
30 def add(request):
31     template = loader.get_template('add.html')
32     return HttpResponseRedirect(template.render({}, request))
33
34
35 def addrecord(request):
36     x = request.POST['first']
37     y = request.POST['last']
38     member = Member(firstname=x, lastname=y)
39     member.save()
40     return HttpResponseRedirect(reverse('members'))
41
```

View
responsável
por salvar o
registro

Adicionando a URL da view

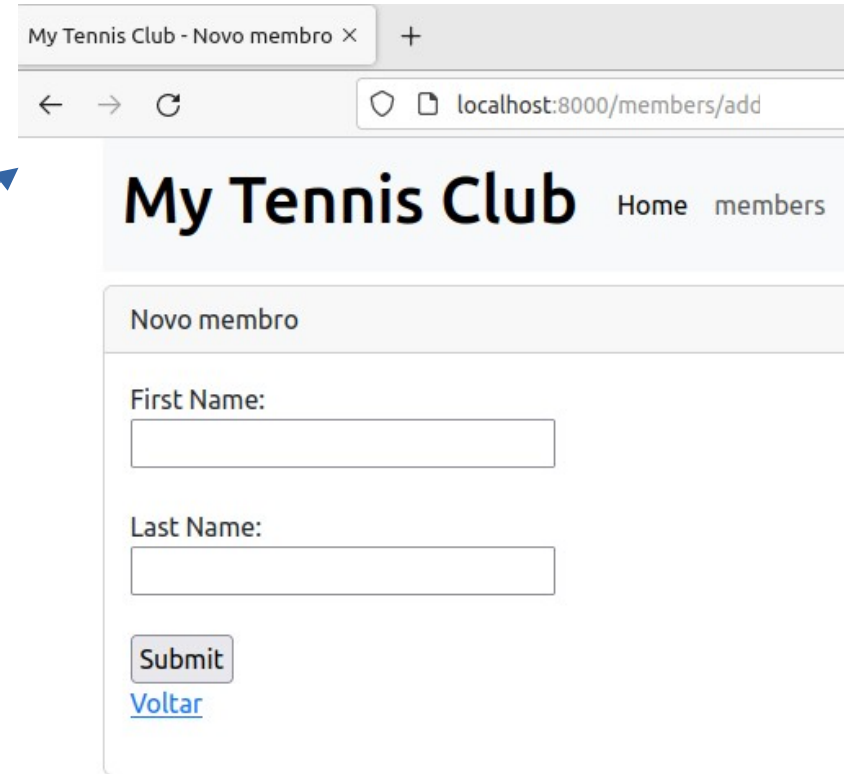
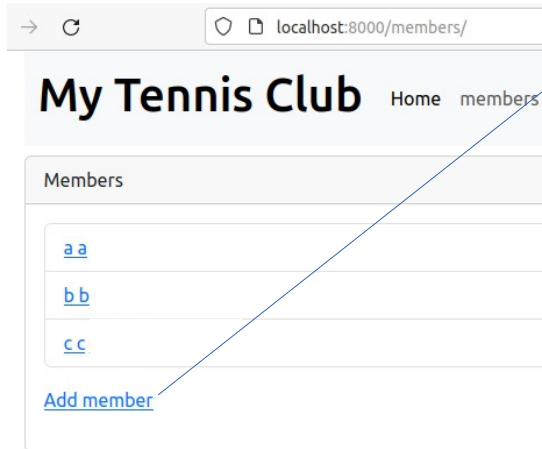
members > urls.py > ...

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.main, name='main'),
6     path('members/', views.members, name='members'),
7     path('members/details/<int:id>', views.details, name='details'),
8     path('members/add/', views.add, name='add'),
9     path('members/add/addrecord/', views.addrecord, name='addrecord'),
10
11 ]
12
```



Testando...

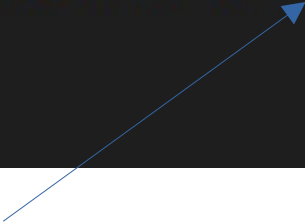
- Agora vai funcionar
- Cadastre novos membros:



3) Delete

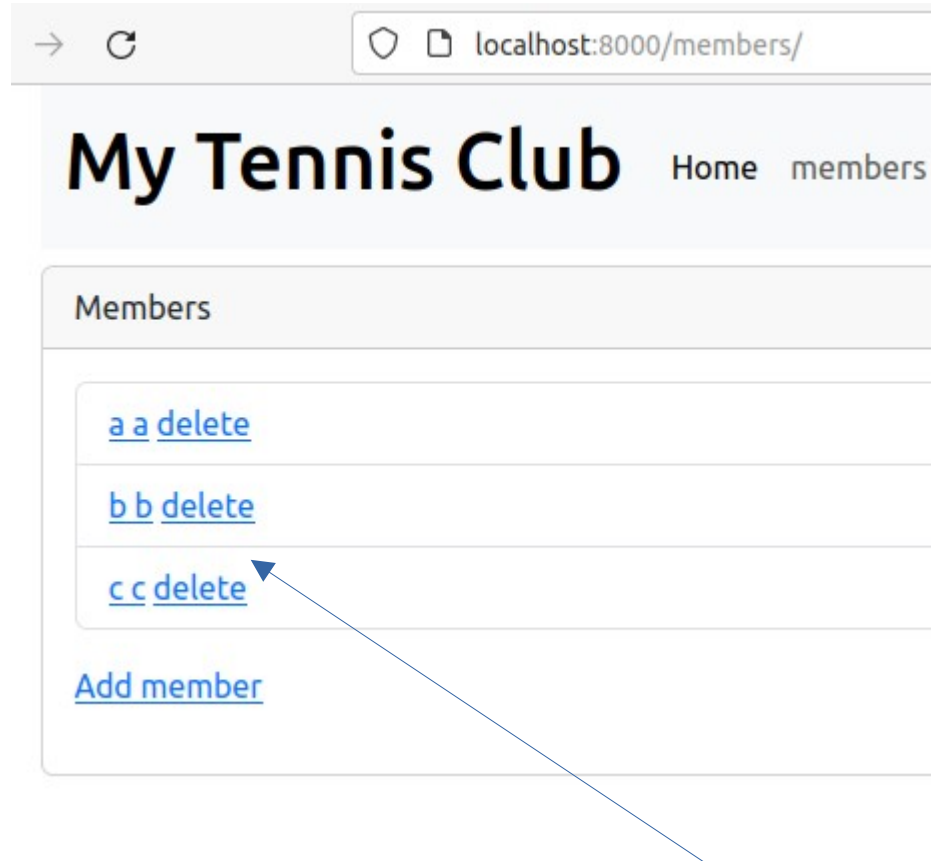
- all_members.html

```
<ul class="list-group">
  {% for x in mymembers %}
  <li class="list-group-item">
    <a href="details/{{ x.id }}">{{ x.firstname }} {{ x.lastname }}</a>
    <a href="delete/{{ x.id }}">delete</a>
  </li>
  {% endfor %}
</ul>
```



Vamos incluir um **link** de exclusão para cada registro

Adicionamos o link em cada item



Trocando o Link por Botão

- Pode ser mais atrativo substituir esse link 'deletar' por um ícone de uma lixeira;
- Vamos utilizar a biblioteca **FontAwesome** para isso;
- Acesse o site e faça um cadastro simples para poder criar um link CDN para seu projeto;
- Os passos a seguir mostram como gerar uma conta e criar o elemento 'script' para incluir no head de seu site.



Start



Icons

Docs

Plans

Support

Blog



Take the hassle out of icons in your **website**.

Font Awesome is the Internet's icon library and toolkit, used by millions of designers, developers, and content creators.



Start for Free



Get More with Pro

Get Started with Font Awesome

The easiest way to get icons on your website is with a Kit. It's your very own custom version of Font Awesome, all bundled up with only the icons, tools, and settings you need.



Enter your email to get started with a free Kit! 

e.g. violet@freshsqueezed.org

Send Kit Embed Code 



Powered by Cloudflare

[Already have an account? Sign In →](#)

WELCOME BACK



Sign In to Your Account

Let's get you signed in and straight to the icons.

Email Address

 jpescola@gmail.com

Password



☒ Remember me

[Forgot your password?](#)

Sign In ➔



Kits

Add New Kit



Last Updated



6 Monthly Pageviews



1cc8d22ca4

FREE

Edited 10 months ago



0

6



Latest Version 6



Open



Web Font



Settings



1cc8d22ca4

FREELatest Version 62 Styles + Brands0 Custom IconsSet UpIconsSettingsOpen6 views

Quick Set Up for the Web

Like your own personal Font Awesome CDN – Just grab a snippet of code, insert it into your HTML or template files, and you're set to add icons to your website, app, or project. We'll host the files and handle the magic.

Download Javascript Sample File

The classic way to use a Font Awesome Kit – just one line of javascript.

```
<script src="https://kit.fontawesome.com/1cc8d22ca4.js" crossorigin="anonymous"></script>
```

1

Copy the unique Kit embed code above and paste it into the `<head>` of your project's HTML file or template.

2

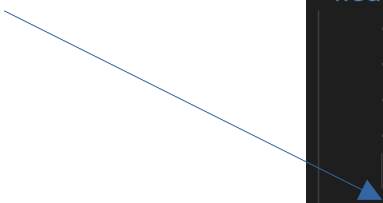
Start placing icons in your HTML's `<body>` using our [simple icon syntax](#).

3

[Color](#), [size](#), [rotate](#), [animate](#), [transform](#) icons with ease!

Inclua o elemento script

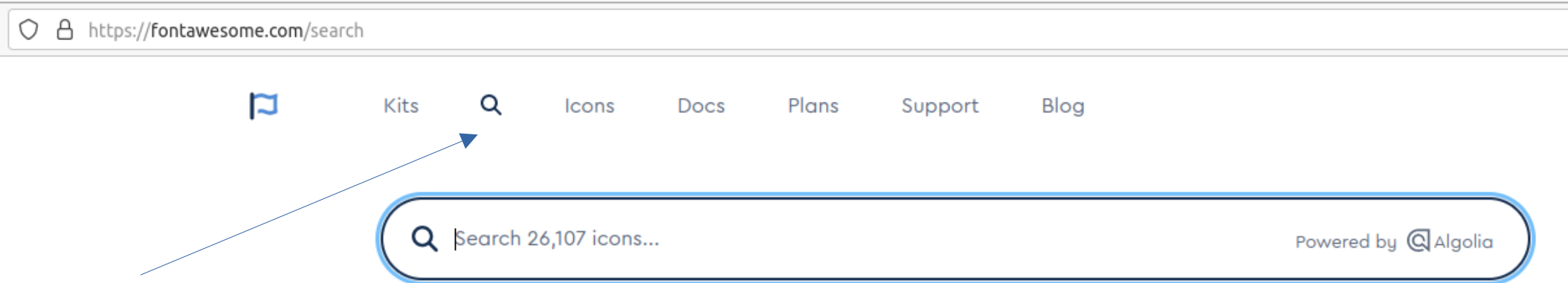
- No head do seu site/sistema, **inclua o elemento 'script'** do slide anterior para instalar o FontAwesome em seu site.
- Agora vamos ver como pesquisar os ícones e encontrar o mais adequado para sua necessidade.



```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My Tennis Club {% block title %}{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css"
        integrity="sha384-GLhlTQ8iRABdZLl603oVMWSktQOp6b7In1Zl3/Jr59b6EGGoIlaFkw7cmDA6j6gD" crossorigin="anonymous">
  <script src="https://kit.fontawesome.com/lcc8d22ca4.js" crossorigin="anonymous"></script>
</head>
```

Pesquisando ícones

- Clique na lupa e digite um termo em inglês:



house 

Unicode: f015

Glyph: 



 [buildings](#) [maps](#)

 Classic 



 No Animation 

 No Rotation 

 Default Size 



Icon Wizard



Get Started

HTML

React

Vue.js

SVG

```
<i class="fa-solid fa-house"></i>
```

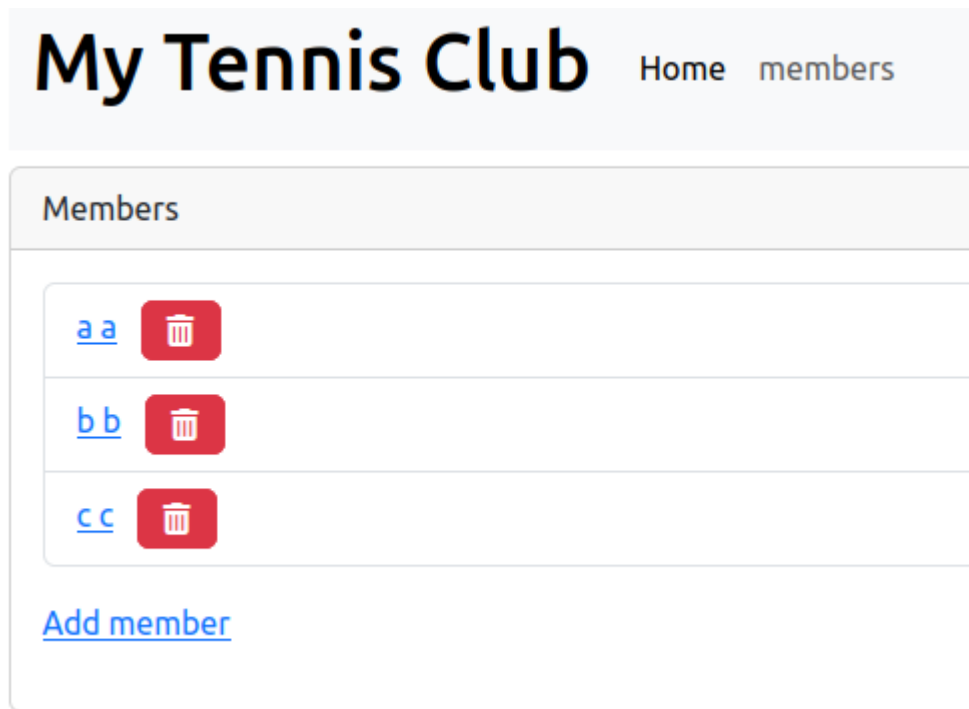
Substituindo o link por button

```
<ul class="list-group">
  {% for x in mymembers %}
  <li class="list-group-item">
    <a href="details/{{ x.id }}">{{ x.firstname }} {{ x.lastname }}</a>
    <a href="delete/{{ x.id }}" class="ms-2 btn btn-danger fa-solid fa-trash-can" ></a>
  </li>
  {% endfor %}
</ul>
```



Ainda não vai funcionar

- Mas já podemos ver o resultado da inclusão da biblioteca e do **novo ícone**:



Adicionando a URL delete

members > urls.py > ...

```
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.main, name='main'),
6      path('members/', views.members, name='members'),
7      path('members/details/<int:id>', views.details, name='details'),
8      path('members/add/', views.add, name='add'),
9      path('members/add/addrecord/', views.addrecord, name='addrecord'),
10     path('members/delete/<int:id>', views.delete, name='delete'),
11 ]
12
```



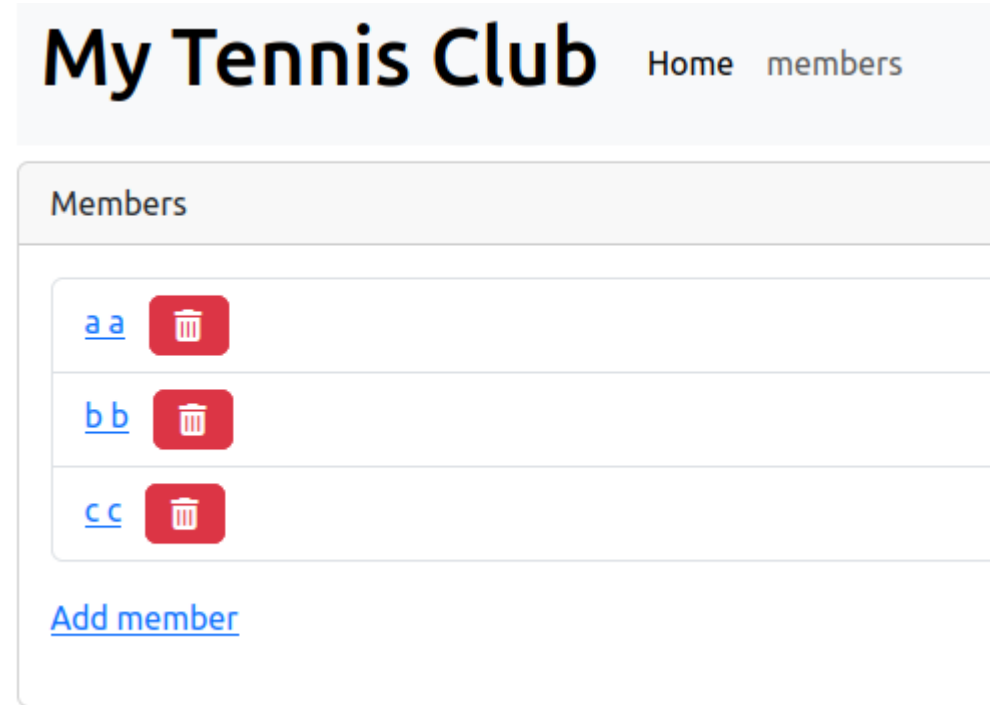
members/views.py

- Adicionando a **view** que recebe o ID, deleta e retorna (reverse) à view members:

```
43 def delete(request, id):
44     member = Member.objects.get(id=id)
45     member.delete()
46     return HttpResponseRedirect(reverse('members'))
47
```

Teste o resultado

- Agora já é possível deletar registros no banco de dados.



4) Update

- all_members.html

```
<ul class="list-group">
  {% for x in mymembers %}
  <li class="list-group-item">
    <a href="details/{{ x.id }}">{{ x.firstname }} {{ x.lastname }}</a>
    <a href="delete/{{ x.id }}" class="btn btn-danger"><i class='fa-solid fa-trash-can'></i></a>
    <a href="update/{{ x.id }}" class="btn btn-info"><i class='fa-solid fa-pen'></i></a>
  </li>
  {% endfor %}
</ul>
```

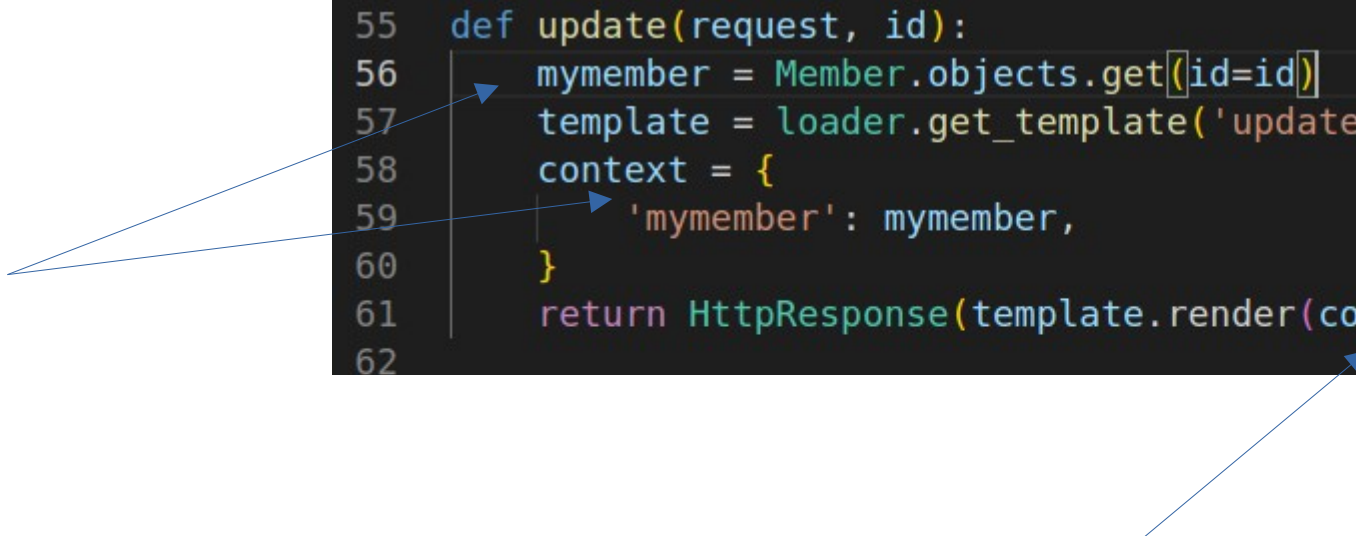
Este link vai carregar o registro no formulário, permitindo alterar algum campo ao invés de criar um novo.

Ícone de alteração

Adicionando a view

- Vamos precisar de um objeto 'member' no **contexto** para preencher o formulário com os dados do membro atual:

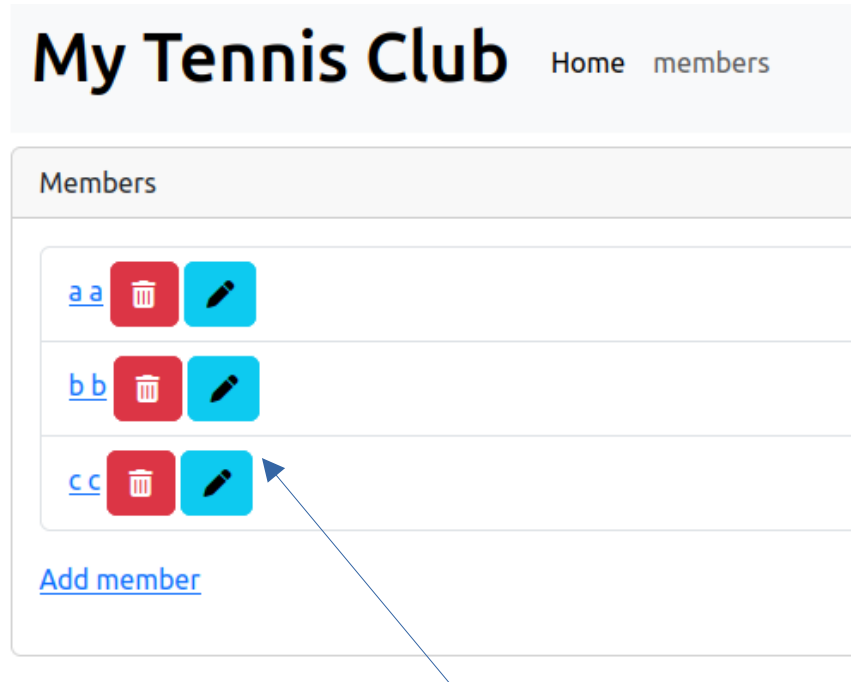
```
55 def update(request, id):
56     mymember = Member.objects.get(id=id)
57     template = loader.get_template('update.html')
58     context = {
59         'mymember': mymember,
60     }
61     return HttpResponse(template.render(context, request))
62
```



The diagram illustrates the data flow in the `update` view function. A blue arrow originates from the `Member.objects.get(id=id)` line (line 56) and points to the `'mymember': mymember` assignment in the `context` dictionary (line 59). Another blue arrow starts from the `context` dictionary and points to the `render(context, request)` call in the `HttpResponse` constructor (line 61). A third blue arrow points from the `render` method to the `HttpResponse` object.

Ainda não vai funcionar

- Mas já podemos ver o ícone de alterar:



```
member templates > update.html
1  {% extends "master.html" %}
2
3  {% block title %}
4  - Atualizando membro
5  {% endblock %}
6
7  {% block content %}
8
9  <div class="card mt-2">
10
11     <div class="card-header">
12         Atualizando dados do membro
13     </div>
14     <div class="card-body">
15
16         <form action="updaterecord/{{ mymember.id }}" method="post">
17             {% csrf_token %}
18             First Name:<br>
19             <input name="first" value="{{ mymember.firstname }}">
20             <br><br>
21             Last Name:<br>
22             <input name="last" value="{{ mymember.lastname }}">
23             <br><br>
24             <input type="submit" value="Submit">
25         </form>
26
27         <p><a href="/members">Voltar</a></p>
28
29     </div>
30 </div>
31
32 {% endblock %}
```

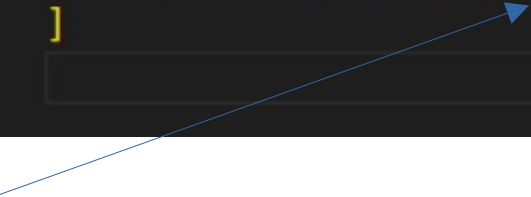
Copie de
add.html

Os campos
virão
preenchidos
com os dados
do objeto
mymember

Url do form de atualização

members >  urls.py > ...

```
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.main, name='main'),
6      path('members/', views.members, name='members'),
7      path('members/details/<int:id>', views.details, name='details'),
8      path('members/add/', views.add, name='add'),
9      path('members/add/addrecord/', views.addrecord, name='addrecord'),
10     path('members/delete/<int:id>', views.delete, name='delete'),
11     path('members/update/<int:id>', views.update, name='update'),
12 ]
13
```



Ainda não vai funcionar

- Mas os dados já são carregados no campo para alteração:

→ ↻ localhost:8000/members/update/1

My Tennis Club

Home members

Atualizando dados do membro

First Name:

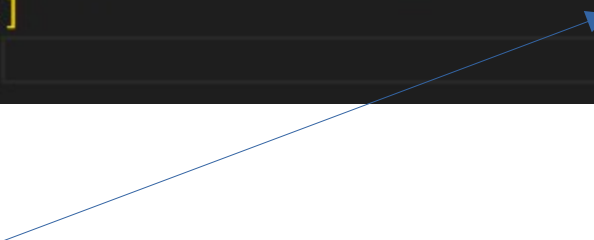
Last Name:

Submit

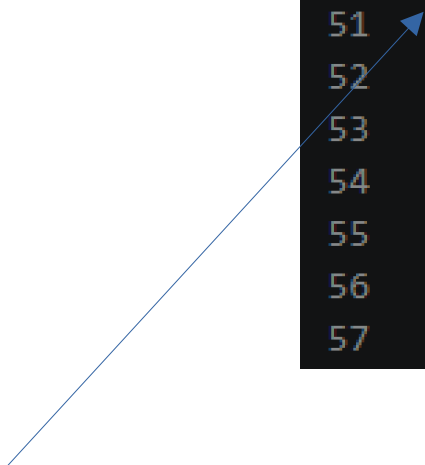
[Voltar](#)

URL para update no BD

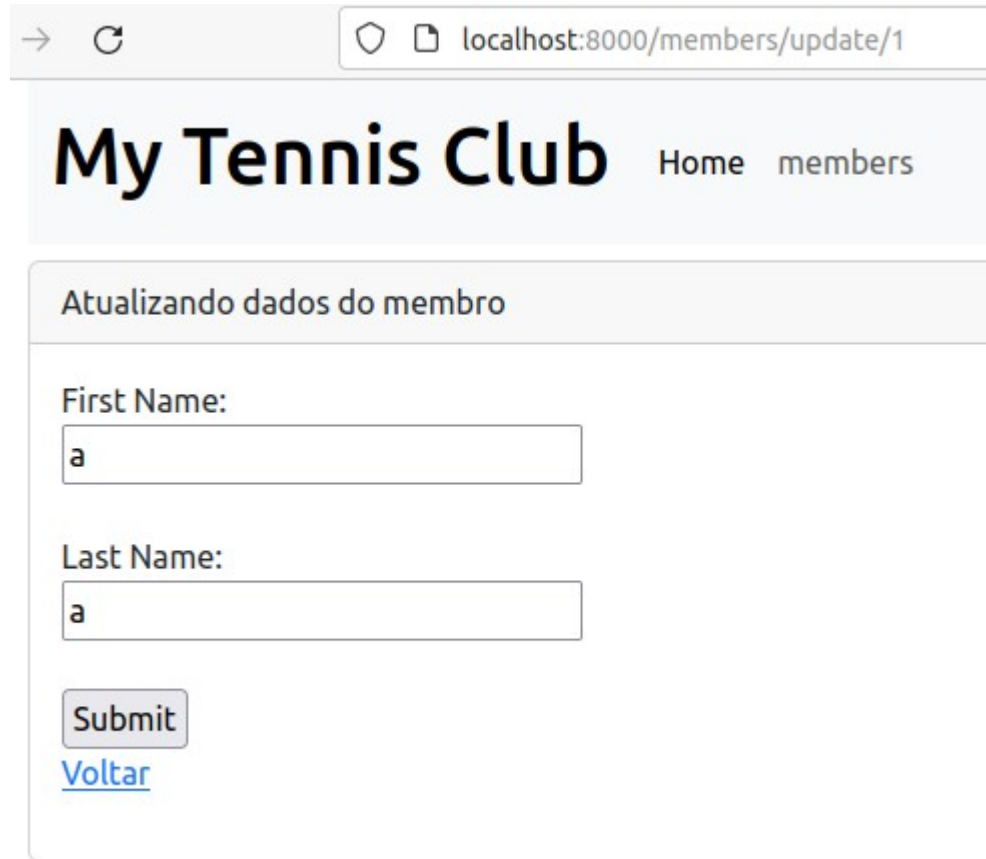
```
members > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.main, name='main'),
6      path('members/', views.members, name='members'),
7      path('members/details/<int:id>', views.details, name='details'),
8      path('members/add/', views.add, name='add'),
9      path('members/add/addrecord/', views.addrecord, name='addrecord'),
10     path('members/delete/<int:id>', views.delete, name='delete'),
11     path('members/update/<int:id>', views.update, name='update'),
12     path('members/update/updaterecord/<int:id>', views.updaterecord, name='updaterecord'),
13 ]
14
```



```
42 def update(request, id):
43     mymember = Member.objects.get(id=id)
44     template = loader.get_template('update.html')
45     context = {
46         'mymember': mymember,
47     }
48     return HttpResponse(template.render(context, request))
49
50 def updaterecord(request, id):
51     first = request.POST['first']
52     last = request.POST['last']
53     member = Member.objects.get(id=id)
54     member.firstname = first
55     member.lastname = last
56     member.save()
57     return HttpResponseRedirect(reverse('members'))
```



Podemos atualizar os membros



The screenshot shows a web browser window with the address bar displaying 'localhost:8000/members/update/1'. The page title is 'My Tennis Club' with navigation links for 'Home' and 'members'. A section titled 'Atualizando dados do membro' contains a form with two text input fields: 'First Name:' and 'Last Name:', both containing the letter 'a'. Below the fields is a 'Submit' button and a blue link labeled 'Voltar'.

→ ↻ localhost:8000/members/update/1

My Tennis Club

Home members

Atualizando dados do membro

First Name:

Last Name:

Submit

[Voltar](#)

Nosso CRUD está completo

- Veja que agora é possível fazer o CRUD completo no nosso sistema;
- Além disso, em aulas anteriores, aprendemos a fazer isso com o **Admin** e com **comandos**;
- Faça os testes:
 - Cadastre os membros:
 - Ayrton Senna da Silva, Francisco Anysio e Pedro Alvares Cabral;
 - Altere o nome dos membros:
 - Ayrton Senna da Silva para Ayrton Senna;
 - Francisco Anysio para Chico Anysio.
 - Exclua o membro Pedro Alvares Cabral.

Exercício

- Crie um novo app chamado **equipments**;
 - O Model deve conter os campos name, register_number e register_date;
 - Crie seu o CRUD completo;
 - Cadastre os equipamentos:
 - Raquete, Rede e Bola.
 - Utilize os PDFs das aulas anteriores como consulta.

Publique seu projeto no GitHub

gh auth login

(acesse a URL e autentique com o código gerado)

git init

git add .

git commit -m “primeiro commit”

gh repo create --public --source=. --push

```
(.venv) C:\Users\aluno\Downloads\my_tennis_club_aula2>gh repo create --public --source=. --push
✓ Created repository jpescola/my_tennis_club_aula2 on github.com
  https://github.com/jpescola/my_tennis_club_aula2
✓ Added remote https://github.com/jpescola/my_tennis_club_aula2.git
Enumerating objects: 34, done.
Counting objects: 100% (34/34), done.
Delta compression using up to 4 threads
Compressing objects: 100% (32/32), done.
Writing objects: 100% (34/34), 8.10 KiB | 592.00 KiB/s, done.
Total 34 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/jpescola/my_tennis_club_aula2.git
 * [new branch]      HEAD -> master
branch 'master' set up to track 'origin/master'.
✓ Pushed commits to https://github.com/jpescola/my_tennis_club_aula2.git

(.venv) C:\Users\aluno\Downloads\my_tennis_club_aula2>
```

O que vimos?

- Integração com Bootstrap;
- Formulários de CRUD;
- Botões de deletar e alterar (FontAwesome).

Referências

- <https://docs.djangoproject.com/en/4.2/>
- <https://fontawesome.com/>
- <https://www.w3schools.com/django>